

1. Write a C# program to Set Up the Development Environment

Solution

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        Console.WriteLine("Hello World!");
```

```
    }
```

```
}
```

2. Write a C# program to Explore Value vs Reference Types

Solution

```
using System;
```

```
class Student
```

```
{
```

```
    public string Name;
```

```
}
```

```
class Program
```

```
{
```

```
static void ModifyValue(int x)
{
    x = 100;
}

static void ModifyReference(Student s)
{
    s.Name = "John";
}

static void Main()
{
    int num = 10;
    Console.WriteLine("Before: " + num);
    ModifyValue(num);
    Console.WriteLine("After: " + num);

    Student st = new Student();
    st.Name = "David";

    Console.WriteLine("Before: " + st.Name);
    ModifyReference(st);
    Console.WriteLine("After: " + st.Name);
}
}
```

3. Write a C# program to Use Primary Constructors in C# 12

Solution

```
using System;
```

```
public class Person(string name, int age)
{
    public string Name { get; } = name;
    public int Age { get; } = age;

    public void Display()
    {
        Console.WriteLine($"Name: {Name}");
        Console.WriteLine($"Age: {Age}");
    }
}
```

```
class Program
{
    static void Main()
    {
        Person p = new Person("Alice", 25);
        p.Display();
    }
}
```

4. Write a C# program to Demonstrate Type Inference with var and new()

Solution

```
using System;
```

```
class Student
```

```
{
```

```
    public string Name { get; set; }
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        var number = 10;
```

```
        var text = "Hello";
```

```
        Student s = new()
```

```
        {
```

```
            Name = "John"
```

```
        };
```

```
        Console.WriteLine(number.GetType());
```

```
    Console.WriteLine(text.GetType());  
    Console.WriteLine(s.GetType());  
  
    Console.WriteLine(number);  
    Console.WriteLine(text);  
    Console.WriteLine(s.Name);  
}  
}
```

5. Write a C# program to Perform Conditional Logic for Grade Calculation
Solution

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        Console.Write("Enter Score: ");  
        int score = Convert.ToInt32(Console.ReadLine());
```

```

if (score >= 90)
    Console.WriteLine("Grade A");
else if (score >= 75)
    Console.WriteLine("Grade B");
else if (score >= 50)
    Console.WriteLine("Grade C");
else
    Console.WriteLine("Fail");

string grade = score switch
{
    >= 90 => "A",
    >= 75 => "B",
    >= 50 => "C",
    _ => "Fail"
};

Console.WriteLine("Switch Grade: " + grade);
}
}

```

6. Write a C# program to Loop Through an Array with Different Loop Types

Solution

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        int[] arr = { 10, 20, 30, 40, 50 };
```

```
        Console.WriteLine("For Loop");
```

```
        for (int i = 0; i < arr.Length; i++)
```

```
        {
```

```
            Console.WriteLine(arr[i]);
```

```
        }
```

```
        Console.WriteLine("Foreach Loop");
```

```
        foreach (int x in arr)
```

```
        {
```

```
            Console.WriteLine(x);
```

```
        }
```

```
        Console.WriteLine("While Loop");
```

```
        int j = 0;
```

```
        while (j < arr.Length)
```

```
        {
```

```
        Console.WriteLine(arr[j]);  
        j++;  
    }  
  
    Console.WriteLine("Do While Loop");  
    int k = 0;  
    do  
    {  
        Console.WriteLine(arr[k]);  
        k++;  
    } while (k < arr.Length);  
}  
}
```

7. Write a C# program to Implement Method Overloading

Solution

```
using System;
```

```
class Program
```

```
{
```

```
    static int CalculateTotal(int a, int b)
```



```
{  
    return a + b;  
}  
  
static double CalculateTotal(double a, double b, double c)  
{  
    return a + b + c;  
}  
  
static void Main()  
{  
    Console.WriteLine(CalculateTotal(10, 20));  
    Console.WriteLine(CalculateTotal(10.5, 20.5, 30.5));  
}  
}
```

8. Write a C# program to Use ref, out, and in Parameters

Solution

```
using System;
```

```
class Program
{
    static void Increment(ref int x)
    {
        x++;
    }

    static void GetValue(out int x)
    {
        x = 100;
    }

    static void Display(in int x)
    {
        Console.WriteLine(x);
    }

    static void Main()
    {
        int a = 10;

        Increment(ref a);
        Console.WriteLine(a);

        int b;
        GetValue(out b);
        Console.WriteLine(b);
    }
}
```

```
        Display(in a);  
    }  
}
```

9. Write a C# program to Use Local Functions

Solution

```
using System;  
  
class Program  
{  
    static int CalculateFactorial(int n)  
    {  
        int Factorial(int x)  
        {  
            if (x <= 1)  
                return 1;  
  
            return x * Factorial(x - 1);  
        }  
    }  
}
```

```
    }

    return Factorial(n);
}

static void Main()
{
    Console.WriteLine(CalculateFactorial(5));
}
}
```

10. Write a C# program to Demonstrate OOP Basics with Constructors

Solution

```
using System;

class Car
{
    public string Make;
```

```
public string Model;
```

```
public int Year;
```

```
public Car()
```

```
{
```

```
    Make = "Unknown";
```

```
    Model = "Unknown";
```

```
    Year = 0;
```

```
}
```

```
public Car(string make, string model, int year)
```

```
{
```

```
    Make = make;
```

```
    Model = model;
```

```
    Year = year;
```

```
}
```

```
public void Display()
```

```
{
```

```
    Console.WriteLine($"{Make} {Model} {Year}");
```

```
}
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
{
```

```
    Car c1 = new Car();
```

```
Car c2 = new Car(  
    "Toyota",  
    "Camry",  
    2024  
);  
  
c1.Display();  
c2.Display();  
}  
}
```

11. Write a C# program to Demonstrate Access Modifiers

Solution

```
using System;
```

```
class BaseClass
```

```
{
```

```
    public string PublicData = "Public Member";
```

```
    private string PrivateData = "Private Member";
```

```
    protected string ProtectedData = "Protected Member";
```

```
public void ShowPrivate()
{
    Console.WriteLine(PrivateData);
}
}
```

```
class DerivedClass : BaseClass
{
    public void Display()
    {
        Console.WriteLine(PublicData);
        Console.WriteLine(ProtectedData);
    }
}
```

```
class Program
{
    static void Main()
    {
        BaseClass obj = new BaseClass();
        Console.WriteLine(obj.PublicData);
        obj.ShowPrivate();

        DerivedClass d = new DerivedClass();
        d.Display();
    }
}
```

12. Write a C# program to Use Auto-Properties and Backing Fields

Solution

```
using System;
```

```
class Product
```

```
{
```

```
    public string Name { get; set; }
```

```
    private double price;
```

```
    public double Price
```

```
    {
```

```
        get { return price; }
```

```
        set
```

```
        {
```

```
            if (value >= 0)
```

```
                price = value;
```

```
            else
```

```
                Console.WriteLine("Price cannot be negative");
```



```

    }
}
}

class Program
{
    static void Main()
    {
        Product p = new Product();

        p.Name = "Laptop";
        p.Price = 50000;

        Console.WriteLine($"{p.Name} - {p.Price}");

        p.Price = -100;
    }
}

```

13. Write a C# program to Create and Use Records with init Properties

Solution

```
using System;
```

```
public record Employee
```

```
{
```

```
    public int Id { get; init; }
```

```
    public string Name { get; init; }
```

```
    public string Department { get; init; }
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        Employee emp1 = new Employee
```

```
        {
```

```
            Id = 1,
```

```
            Name = "John",
```

```
            Department = "IT"
```

```
        };
```

```
        Employee emp2 = emp1 with
```

```
        {
```

```
            Name = "David"
```

```
        };
```

```
        Console.WriteLine(emp1);
```

```
        Console.WriteLine(emp2);
```

```
}  
}
```

14. Write a C# program to Demonstrate Inheritance and Method Overriding
Solution

```
using System;
```

```
class Shape
```

```
{  
    public virtual void Draw()  
    {  
        Console.WriteLine("Drawing Shape");  
    }  
}
```

```
class Circle : Shape
```

```
{  
    public override void Draw()  
    {  
        Console.WriteLine("Drawing Circle");  
    }  
}
```

```
class Rectangle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing Rectangle");
    }
}
```

```
class Program
{
    static void Main()
    {
        Shape s1 = new Circle();
        Shape s2 = new Rectangle();

        s1.Draw();
        s2.Draw();
    }
}
```

15. Write a C# program to Differentiate Abstract Classes and Interfaces

Solution

```
using System;
```

```
abstract class Vehicle
```

```
{  
    public abstract void Drive();  
}
```

```
interface IDrivable
```

```
{  
    void Start();  
}
```

```
class Car : Vehicle, IDrivable
```

```
{  
    public override void Drive()  
    {  
        Console.WriteLine("Car is Driving");  
    }
```

```
    public void Start()
```

```
    {  
        Console.WriteLine("Car Started");  
    }  
}
```

```
class Program
```

```
{
```

```
static void Main()
{
    Vehicle v = new Car();
    v.Drive();

    IDrivable d = new Car();
    d.Start();
}
}
```

16. Write a C# program to Handle Null References Safely
Solution

```
using System;

class Person
{
    public string? Name { get; set; }
}

class Program
{
    static void Main()
    {
```

```
Person? person = null;

Console.WriteLine(person?.Name ?? "No Name");

person = new Person();
Console.WriteLine(person?.Name ?? "Name Missing");
}
}
```

17. Write a C# program to Use Null-Conditional Chaining in a Contact App
Solution

```
using System;

class Contact
{
    public string? Name { get; set; }
    public string? PhoneNumber { get; set; }
}

class Program
{
    static void Main()
    {
```

```

Contact? contact = null;

Console.WriteLine(
    contact?.Name ?? "Contact Not Found"
);

contact = new Contact
{
    Name = "John",
    PhoneNumber = "9876543210"
};

Console.WriteLine(
    contact?.Name ?? "Contact Not Found"
);
}
}

```

18. Write a C# program to Use the required Modifier in C# 12

Solution

```
using System;
```



```
class Student
{
    public required string Name { get; set; }
    public required int RollNo { get; set; }
}
```

```
class Program
{
    static void Main()
    {
        Student s = new Student
        {
            Name = "Alice",
            RollNo = 101
        };

        Console.WriteLine(
            $"{s.Name} - {s.RollNo}"
        );
    }
}
```

19. Write a C# program to Work with Lists and Dictionaries

Solution

```
using System;

using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<string> fruits = new List<string>()
        {
            "Apple",
            "Banana",
            "Mango"
        };

        fruits.Add("Orange");
        fruits.Remove("Banana");

        Console.WriteLine("List Items");

        foreach(string item in fruits)
        {
            Console.WriteLine(item);
        }
    }
}
```

```
Dictionary<int,string> students =  
    new Dictionary<int,string>();  
  
students.Add(1,"John");  
students.Add(2,"David");  
students.Add(3,"Alice");  
  
students.Remove(2);  
  
Console.WriteLine("\nDictionary Items");  
  
foreach(var item in students)  
{  
    Console.WriteLine(  
        item.Key + " " + item.Value  
    );  
}  
}
```

20. Write a C# program to Use LINQ for Filtering and Projection

Solution

```
using System;

using System.Collections.Generic;

using System.Linq;

class Order
{
    public int OrderId { get; set; }

    public string CustomerName { get; set; }

    public double TotalAmount { get; set; }
}

class Program
{
    static void Main()
    {
        List<Order> orders = new List<Order>()
        {
            new Order
            {
                OrderId = 1,
                CustomerName = "John",
                TotalAmount = 1000
            }
        }
    }
}
```

```
},
```

```
new Order
```

```
{
```

```
    OrderId = 2,
```

```
    CustomerName = "David",
```

```
    TotalAmount = 5000
```

```
},
```

```
new Order
```

```
{
```

```
    OrderId = 3,
```

```
    CustomerName = "Alice",
```

```
    TotalAmount = 3000
```

```
}
```

```
};
```

```
var result =
```

```
    orders
```

```
    .Where(o => o.TotalAmount > 2000)
```

```
    .Select(o => new
```

```
{
```

```
    o.OrderId,
```

```
    o.CustomerName
```

```
});
```

```
foreach(var item in result)
```

```
{
```

```
        Console.WriteLine(
            $"{item.OrderId}{item.CustomerName}"
        );
    }
}
}
```

21. Write a C# program to Use Pattern Matching with is and switch

Solution

```
using System;
```

```
class Program
```

```
{
    static void DisplayInfo(object obj)
    {
        if (obj is int number)
        {
            Console.WriteLine($"Integer: {number}");
        }
        else if (obj is string text)
        {
            Console.WriteLine($"String: {text}");
        }
    }
}
```

```
switch (obj)
{
    case int n:
        Console.WriteLine($"Square: {n * n}");
        break;

    case string s:
        Console.WriteLine($"Length: {s.Length}");
        break;

    default:
        Console.WriteLine("Unknown Type");
        break;
}

static void Main()
{
    DisplayInfo(10);
    DisplayInfo("Hello");
}
```

22. Write a C# program to Create and Deconstruct Tuples

Solution

```
using System;

class Program
{
    static (int, string) GetData()
    {
        return (101, "John");
    }

    static void Main()
    {
        var (id, name) = GetData();

        Console.WriteLine($"ID: {id}");
        Console.WriteLine($"Name: {name}");
    }
}
```

23. Write a C# program to Simulate Async File Upload with Exception Handling

Solution


```
using System;

using System.Threading.Tasks;

class Program
{
    static async Task<string> UploadFileAsync()
    {
        await Task.Delay(3000);
        return "File Uploaded Successfully";
    }

    static async Task Main()
    {
        try
        {
            string result = await UploadFileAsync();
            Console.WriteLine(result);
        }
        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }
}
```

24. Write a C# program to Serialize and Deserialize JSON Files

Solution

```
using System;

using System.IO;

using System.Text.Json;

class User
{
    public string Name { get; set; }
    public int Age { get; set; }
    public string Email { get; set; }
}

class Program
{
    static void Main()
    {
        User user = new User
        {
            Name = "John",
            Age = 25,
            Email = "john@gmail.com"
        };

        string json =
```

```
        JsonSerializer.Serialize(user);

        File.WriteAllText("user.json", json);

        string data =
            File.ReadAllText("user.json");

        User result =
            JsonSerializer.Deserialize<User>(data);

        Console.WriteLine(result.Name);
        Console.WriteLine(result.Age);
        Console.WriteLine(result.Email);
    }
}
```

25. Write a C# program to Use FileStream and MemoryStream

Solution

```
using System;
using System.IO;
using System.Text;

class Program
```

```
{
    static void Main()
    {
        using(FileStream fs =
            new FileStream(
                "sample.txt",
                FileMode.OpenOrCreate))
        {
            byte[] data =
                Encoding.UTF8.GetBytes(
                    "Hello FileStream");

            fs.Write(data,0,data.Length);
        }

        using(FileStream fs =
            new FileStream(
                "sample.txt",
                FileMode.Open))
        {
            byte[] buffer =
                new byte[fs.Length];

            fs.Read(buffer,0,buffer.Length);

            Console.WriteLine(
                Encoding.UTF8.GetString(buffer));
        }
    }
}
```

```
MemoryStream ms =  
    new MemoryStream();  
  
byte[] bytes =  
    Encoding.UTF8.GetBytes(  
        "Memory Stream Data");  
  
ms.Write(bytes,0,bytes.Length);  
  
Console.WriteLine(  
    "Bytes Written: " +  
    ms.Length);  
}  
}
```

26. Write a C# program to Demonstrate Race Conditions with Multi-threading
Solution

```
using System;  
using System.Threading;  
  
class Program  
{
```

```
static int counter = 0;
```

```
static readonly object lockObj =  
    new object();
```

```
static void Increment()  
{  
    for(int i=0;i<10000;i++)  
    {  
        lock(lockObj)  
        {  
            counter++;  
        }  
    }  
}
```

```
static void Main()  
{  
    Thread t1 =  
        new Thread(Increment);  
  
    Thread t2 =  
        new Thread(Increment);  
  
    t1.Start();  
    t2.Start();  
  
    t1.Join();
```

```

t2.Join();

Console.WriteLine(
    "Counter = " + counter);
}
}

```

27. Write a C# program to Simulate and Resolve a Deadlock Solution

```

using System;
using System.Threading;

class Program
{
    static object lock1 = new object();
    static object lock2 = new object();

    static void Task1()
    {
        bool acquired =
            Monitor.TryEnter(lock1, 1000);

        if(acquired)

```

```

    {
        try
        {
            Console.WriteLine(
                "Task1 acquired Lock1");
        }
        finally
        {
            Monitor.Exit(lock1);
        }
    }
}

static void Main()
{
    Thread t1 =
        new Thread(Task1);

    t1.Start();
    t1.Join();
}
}

```

28. Write a C# program to Log with System.Diagnostics.Trace

Solution


```
using System;
using System.Diagnostics;

class Program
{
    static void Main()
    {
        Trace.Listeners.Add(
            new TextWriterTraceListener(
                "log.txt"));

        Trace.AutoFlush = true;

        Trace.WriteLine(
            "Application Started");

        Console.WriteLine(
            "Log Written");

        Trace.WriteLine(
            "Application Ended");
    }
}
```

29. Write a C# program to Sanitize Input and Prevent XSS

Solution

```
using System;
using System.Net;

class Program
{
    static void Main()
    {
        Console.Write(
            "Enter Input: ");

        string input =
            Console.ReadLine();

        string safeInput =
            WebUtility.HtmlEncode(input);

        Console.WriteLine(
            "Sanitized Output:");

        Console.WriteLine(
            safeInput);
    }
}
```

30. Write a C# program to Perform CRUD Operations using ADO.NET

Solution

```
using System;

using System.Data.SqlClient;

class Program
{
    static string connectionString =
        @"Server=localhost;
        Database=CompanyDB;
        Trusted_Connection=True;";

    static void Main()
    {
        InsertEmployee();
        ReadEmployees();
        UpdateEmployee();
        DeleteEmployee();
    }

    static void InsertEmployee()
    {
```

```
using(SqlConnection con =  
    new SqlConnection(connectionString))  
{  
    con.Open();  
  
    string query =  
        "INSERT INTO Employees(Name,Salary) " +  
        "VALUES('John',50000)";  
  
    SqlCommand cmd =  
        new SqlCommand(query,con);  
  
    cmd.ExecuteNonQuery();  
  
    Console.WriteLine(  
        "Record Inserted");  
}  
}
```

```
static void ReadEmployees()  
{  
    using(SqlConnection con =  
        new SqlConnection(connectionString))  
    {  
        con.Open();  
  
        string query =  
            "SELECT * FROM Employees";
```

```

SqlCommand cmd =
    new SqlCommand(query,con);

SqlDataReader reader =
    cmd.ExecuteReader();

while(reader.Read())
{
    Console.WriteLine(
        reader["Id"] + " " +
        reader["Name"] + " " +
        reader["Salary"]);
}
}

static void UpdateEmployee()
{
    using(SqlConnection con =
        new SqlConnection(connectionString))
    {
        con.Open();

        string query =
            "UPDATE Employees " +
            "SET Salary=60000 " +
            "WHERE Name='John'";

```

```
SqlCommand cmd =  
    new SqlCommand(query,con);  
  
cmd.ExecuteNonQuery();  
  
Console.WriteLine(  
    "Record Updated");  
}  
}  
  
static void DeleteEmployee()  
{  
    using(SqlConnection con =  
        new SqlConnection(connectionString))  
    {  
        con.Open();  
  
        string query =  
            "DELETE FROM Employees " +  
            "WHERE Name='John'";  
  
        SqlCommand cmd =  
            new SqlCommand(query,con);  
  
        cmd.ExecuteNonQuery();  
  
        Console.WriteLine(  

```

```
        "Record Deleted");  
    }  
}  
}
```